Processador RISC-V RV32I de ciclo único

Relatório final

Gabriel Sena Barbosa Silva
Jair Rocha Souza
João Vítor Cedraz Carneiro
Luiz Augusto dos Santos Oliveira
Tássio Carvalho Rodrigues

Programa CI Digital — CEPEDI

31 de outubro de 2026

Repositório: `Gabriel-SBS/rv32i-processor`

Sumário

1 Arquitetura **1**
1.1 Clock e reset . . . 2
1.2 Busca . . . 2
1.3 Decodificação . . . 3
1.4 Execução . . . 3
1.5 Memória . . . 4
1.6 Write-back . . . 5

2 Instruções implementadas **5**

3 Verificação **5**

4 Resultados **6**
4.1 Testes de cada bloco . . . 6
4.2 Programas no processador completo . . . 7
4.3 Erros encontrados pelos programas . . . 7

5 Formas de onda **7**

6 Conclusão **13**

1 Arquitetura

O processador implementa o conjunto de instruções base RV32I, de 32 bits, em ciclo único: cada instrução é buscada, decodificada, executada, acessa a memória quando precisa e grava o resultado no banco de registradores dentro de um único ciclo de clock.

Para explicar o caminho de dados, dividimos o processador nos cinco estágios da Figura 1. Não há registradores entre eles, porque o processador é de ciclo único; a divisão é só para organizar a explicação. Nas figuras de cada estágio, os blocos com contorno contínuo pertencem

ao estágio, e os de contorno tracejado são sinais que vêm de outros estágios ou vão para eles. Os códigos Mermaid estão em `relatorio/figuras/arq_*.mmd`.

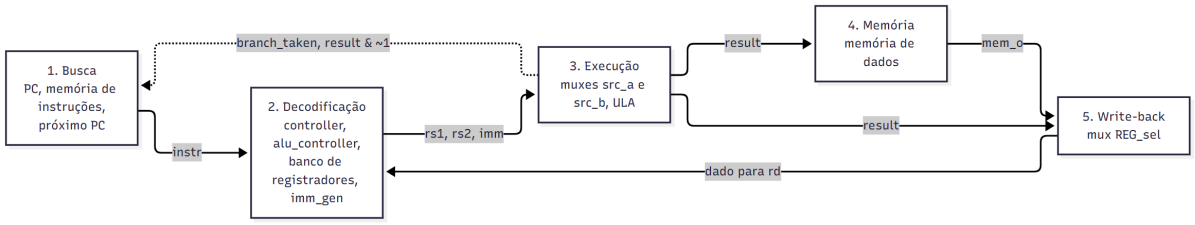


Figura 1: Visão geral: os cinco estágios e o que passa entre eles. A linha pontilhada é a volta para a busca, usada nos desvios e no JALR.

1.1 Clock e reset

O reset `rst_n` é ativo em nível baixo e assíncrono. Com `rst_n = 0`, o PC vai para `0x00000000` na hora, sem esperar o clock. As escritas acontecem na borda de subida de `clk`. O registrador `x0` vale sempre zero: escritas nele são ignoradas.

1.2 Busca

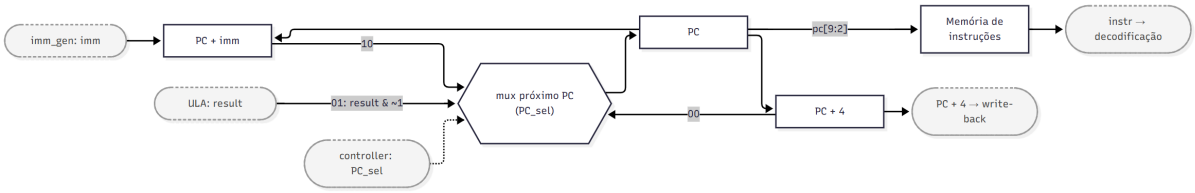


Figura 2: Busca da instrução e cálculo do próximo PC.

O PC endereça a memória de instruções, que tem 256 palavras de 32 bits, lidas de `instructions/instruction_control.hex` no início da simulação. O endereço é `pc[9:2]`, então os bits 1:0 são ignorados. No `tb_processor`, o programa escolhido com `+TEST=<nome>` é carregado por cima desse arquivo.

Ao mesmo tempo, o `pc_logic` calcula os candidatos a próximo PC, e um `mux4` escolhe entre eles pelo sinal `PC_sel`:

- `00`: `PC + 4`, na execução normal e quando o desvio não é tomado;
- `01`: `(rs1 + imm) & ~1`, no `JALR`, com o resultado vindo da ULA;
- `10`: `PC + imm`, nos desvios tomados e no `JAL`.

A seleção `11` não é usada e também dá `PC + 4`. O valor `PC + 4` vai ainda para o write-back, porque é o endereço de retorno que o JAL e o JALR gravam em `rd`.

1.3 Decodificação

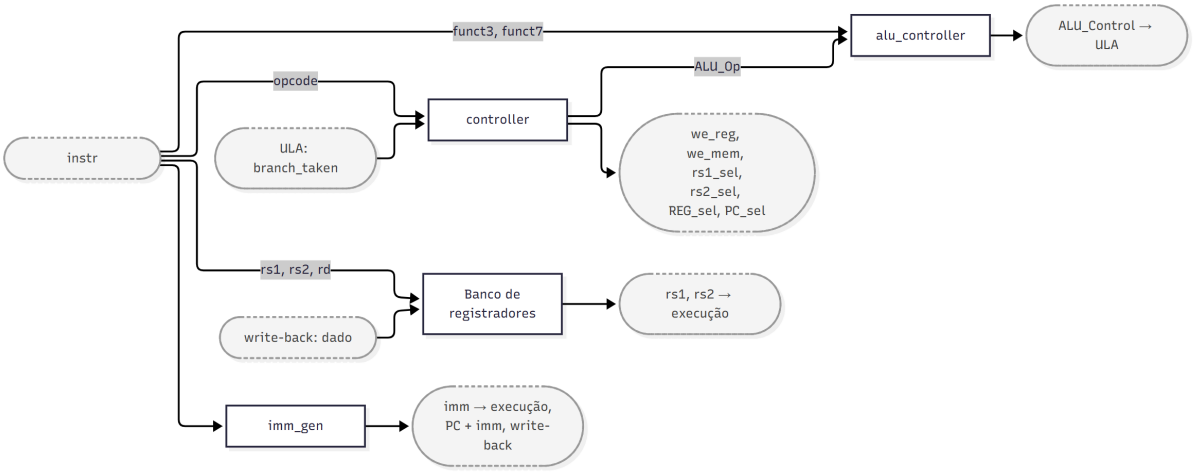


Figura 3: Decodificação. Cada campo da instrução vai para um bloco diferente.

A instrução é quebrada em campos, e cada campo vai para um bloco:

- `controller` lê o `opcode` e gera `we_reg`, `we_mem`, `rs1_sel`, `rs2_sel`, `REG_sel`, `PC_sel` e `ALU_Op`. Nos desvios, quem diz se o salto acontece é a saída `branch_taken` da ULA: com ela em 1, o `controller` põe `PC_sel = 10`.
- `alu_controller` traduz `ALU_Op`, `funct3` e `funct7[5]` para o código de operação da ULA.
- O banco de registradores tem 32 registradores de 32 bits, duas portas de leitura combinacional (`rs1` e `rs2`) e uma de escrita (`rd`), que só grava na borda de subida e com `we_reg = 1`.
- `imm_gen` monta o imediato dos formatos I, S, B, U e J, com extensão de sinal.

1.4 Execução

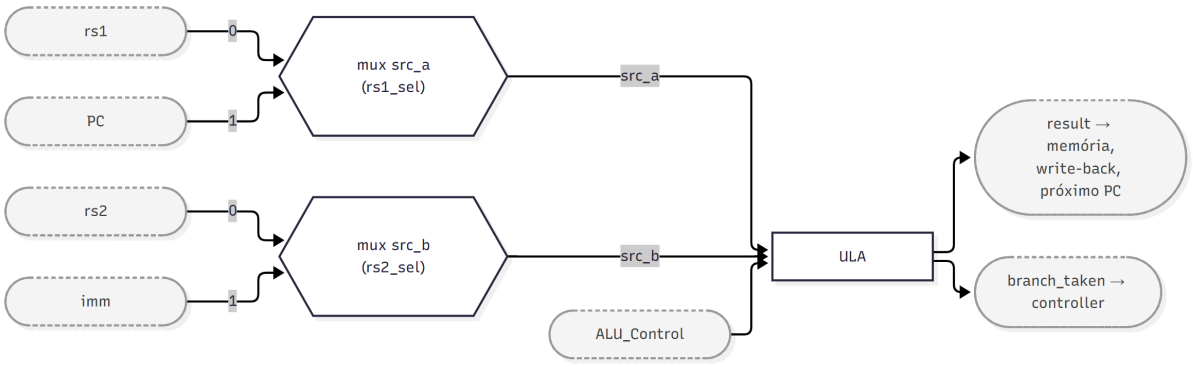


Figura 4: Escolha dos operandos e operação na ULA.

Dois muxes escolhem as entradas da ULA. O `src_a` é `rs1`, ou o PC quando `rs1_sel = 1` (AUIPC e JAL). O `src_b` é `rs2`, ou o imediato quando `rs2_sel = 1` (instruções do tipo I, loads, stores, LUI, AUIPC, JAL e JALR).

A ULA é combinacional, com a interface `ALU (a, b, ALUControl, result, branch_taken)`. A codificação segue o próprio RISC-V: nas instruções do tipo R, `ALUControl = {funct7[5], funct3}`. Os 16 códigos são usados (Tabela 1).

Tabela 1: Códigos de operação da ULA.

`ALUControl`	Operação	`ALUControl`	Operação
`0000`	ADD	`1000`	SUB
`0001`	SLL	`1001`	BEQ (`a == b`)
`0010`	SLT (com sinal)	`1010`	BNE (`a != b`)
`0011`	SLTU (sem sinal)	`1011`	BGE (com sinal)
`0100`	XOR	`1100`	BLT (com sinal)
`0101`	SRL	`1101`	SRA
`0110`	OR	`1110`	BLTU (sem sinal)
`0111`	AND	`1111`	BGEU (sem sinal)

Os deslocamentos usam só `b[4:0]`. As comparações devolvem 1 ou 0 em `result`. A saída `branch_taken` só sobe nos seis códigos de desvio, e apenas quando a condição é verdadeira. Num `SLT` que dá 1, por exemplo, ela continua em 0, então nenhuma instrução que não seja desvio consegue mudar o fluxo do programa.

O `result` vai para a memória de dados, como endereço, e para o write-back. No JALR, ele também é o destino do salto.

1.5 Memória

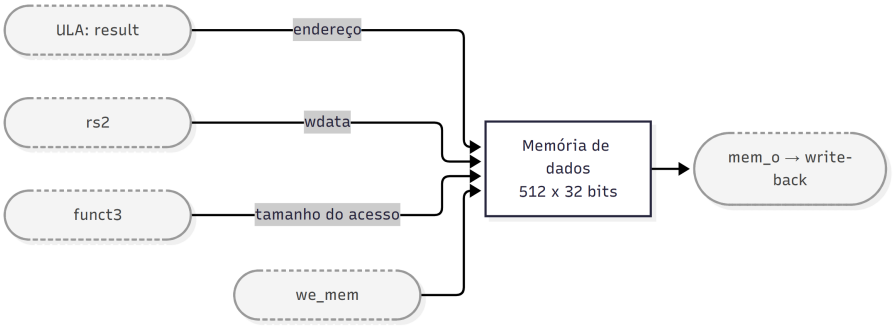


Figura 5: Memória de dados e os sinais que ela recebe.

A memória de dados tem 512 palavras de 32 bits (2 KiB). O endereço, em bytes, vem do `result` da ULA, e o dado a gravar vem de `rs2`. O `funct3` da instrução diz o tamanho do acesso: byte (`sb`, `lb`, `lbu`), meia palavra (`sh`, `lh`, `lhu`) ou palavra (`sw`, `lw`). A escrita é síncrona, só com `we_mem = 1`, e muda apenas os bytes do acesso. A leitura é combinacional: `lb` e `lh` estendem o sinal, e `lbu` e `lhu` completam com zeros.

1.6 Write-back

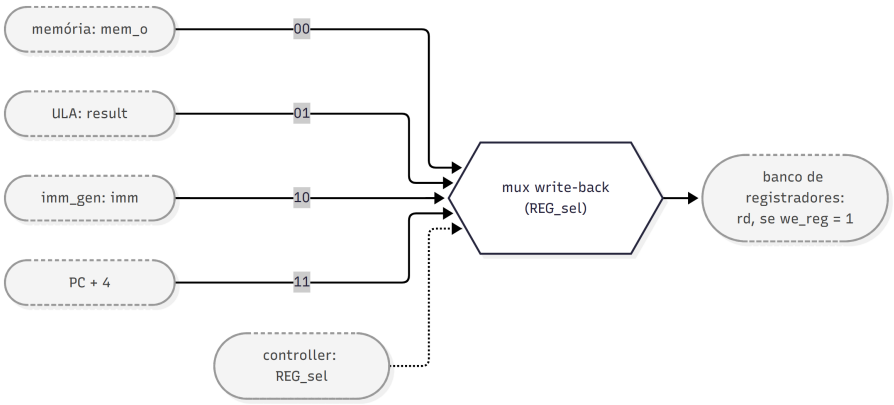


Figura 6: Mux de write-back, que escolhe o valor gravado em `rd`.

O último mux escolhe, pelo `REG_sel`, o que será gravado em `rd`:

- `00`: o dado lido da memória, nos loads;
- `01`: o `result` da ULA, nas instruções dos tipos R e I e no AUIPC;
- `10`: o imediato, no LUI;
- `11`: `PC + 4`, no JAL e no JALR.

Stores e desvios não gravam nada. Nesses casos o `controller` deixa `we_reg = 0` e a saída do mux é ignorada.

2 Instruções implementadas

Tabela 2: Instruções do RV32I implementadas e testadas.

Grupo	Formato	Instruções
Aritméticas e lógicas entre registradores	R	ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU
Aritméticas e lógicas com imediato	I	ADDI, ANDI, ORI, XORI, SLLI, SRLI, SRAI, SLTI, SLTIU
Loads	I	LB, LH, LW, LBU, LHU
Stores	S	SB, SH, SW
Desvios condicionais	B	BEQ, BNE, BLT, BGE, BLTU, BGEU
Saltos	J / I	JAL, JALR
Imediato superior	U	LUI, AUIPC

3 Verificação

A verificação teve duas partes.

Primeiro, cada bloco foi testado sozinho. São 14 testbenches, que combinam casos escritos à mão (valores-limite, estouro, reset, `x0`) com milhares de entradas aleatórias comparadas a um modelo de referência escrito dentro do próprio testbench.

Depois, o processador inteiro executou 10 programas em assembly RV32I. Cada programa termina num laço `fim: j fim`. Quando o processador chega nesse laço, o `tb_processor` confere:

- se `x31 = 1`, valor que o próprio programa grava quando todos os seus subtestes passam;
- se o PC parou no endereço esperado;
- se os 32 registradores têm os mesmos valores calculados pelo simulador de referência `rv32i_sim.py`.

Os programas usam `x30` para guardar o número do subteste em andamento. Se algum falhar, o programa grava `0xBAD` em `x31`, e `x30` mostra em qual subteste parou.

O código-fonte dos programas (`.s`), o montador `rv32i_asm.py`, o simulador `rv32i_sim.py` e o script `gerar_todos.sh`, que gera os `.hex` e os `.regs`, estão em `instructions/arquivos_assembly/Treino_RISC_V/testes/`.

Todos os testes rodam com `sim/run_xrun.sh` no Cadence Xcelium 25.03.

4 Resultados

4.1 Testes de cada bloco

Tabela 3: Testbenches dos blocos. Todos passaram.

Bloco	Testbench	Verif.	O que é testado
Somador	`tb_adder`	8	PC + 4, imediato positivo e negativo, estouro de 32 bits
Mux 2:1	`tb_mux2`	207	as duas seleções, 200 casos aleatórios
Mux 4:1	`tb_mux4`	7	as quatro seleções
PC	`tb_pc`	11	reset assíncrono, carga só na borda de subida
Próximo PC	`tb_pc_logic`	18	sequencial, desvio tomado e não tomado, JAL, JALR
ULA	`tb_alu`	51	as 16 operações, os 6 desvios, `branch_taken = 0` fora dos desvios
ULA contra modelo	`tb_alu_ref`	23.400	3.400 casos de borda e 20.000 aleatórios
Banco de registradores	`tb_register_file`	1.214	duas portas de leitura, `x0`, `we = 0`, 500 aleatórios
Banco + ULA	`tb_regfile_alu`	5.168	instruções que usam o resultado da anterior, 5.000 aleatórias
Memória de dados	`tb_data_memory`	3.559	byte, meia palavra e palavra, 2.000 acessos aleatórios
Memória de instruções	`tb_instruction_memory`	46	leitura alinhada, posições vazias, endereço além do fim
Gerador de imediatos	`tb_imm_gen`	4.526	formatos I, S, B, U e J, 500 casos por opcode
Controle da ULA	`tb_alu_controller`	329	todas as combinações de `ALU_Op`, `funct3` e `funct7[5]`
Unidade de controle	`tb_controller`	274	9 classes de instrução, os 128 opcodes
Total		38.818	

4.2 Programas no processador completo

Tabela 4: Programas executados no processador. Todos passaram, com o mesmo número de ciclos do simulador de referência.

Programa	Ciclos	Verif.	O que exercita
01 `alu_reg`	70	34	operações entre registradores
02 `alu_imm`	64	34	operações com imediato
03 `shifts`	77	34	deslocamentos lógicos e aritméticos
04 `comparacoes`	51	34	comparações com e sem sinal
05 `lui_auipc`	55	34	LUI e AUIPC
06 `load_store_word`	84	34	LW e SW com vários offsets
07 `load_store_byte_half`	90	34	LB, LH, LBU, LHU, SB, SH
08 `branches`	113	34	os seis desvios condicionais
09 `jal_jalr`	66	34	JAL, JALR e endereço de retorno
10 `algoritmo_final`	723	34	soma de vetor, Fibonacci, bubble sort, contagem de bits e tamanho de string
Total	1.393	340	

4.3 Erros encontrados pelos programas

Os blocos passavam nos testes individuais, mas três programas acharam erros que só aparecem com tudo ligado:

Tabela 5: Erros encontrados e correções.

Programa	O que acontecia	Correção
05 `lui_auipc`	O AUIPC dava `PC + 2·imm`: o mux de `src_a` recebia `pc_target`, que já é `PC + imm`, em vez de `pc`	`src_a` passou a receber `pc` quando `rs1_sel = 1`
07 `load_store_byte_half`	LB, LH, LBU, LHU, SB e SH não funcionavam: a memória só lia e escrevia palavras inteiras	a memória passou a receber `funct3` e o endereço em bytes
10 `algoritmo_final`	Com 64 palavras, os endereços `0x400` e `0x500` caíam na mesma posição	memória de 512 palavras, parâmetro `DM_ADDR_W = 9`

5 Formas de onda

As figuras foram geradas com o script `sim/figuras_xrun.sh`. Ele simula cada programa no `xrun` e para a simulação com um breakpoint condicional na instrução de interesse (por exemplo, "opcode igual a AUIPC"). A parada acontece na borda de descida do clock, no meio do ciclo, quando todos os sinais já estão estáveis.

```
sim/figuras_xrun.sh # simula todas as figuras
sim/figuras_xrun.sh view fig04 # abre a fig04 com o cursor no ponto
```

Os sinais aparecem em grupos: clock e reset, busca, decodificação, controle, execução, memória e status do teste (`x30` e `x31`). O clock tem período de 10 ns e o reset é liberado em 20 ns.

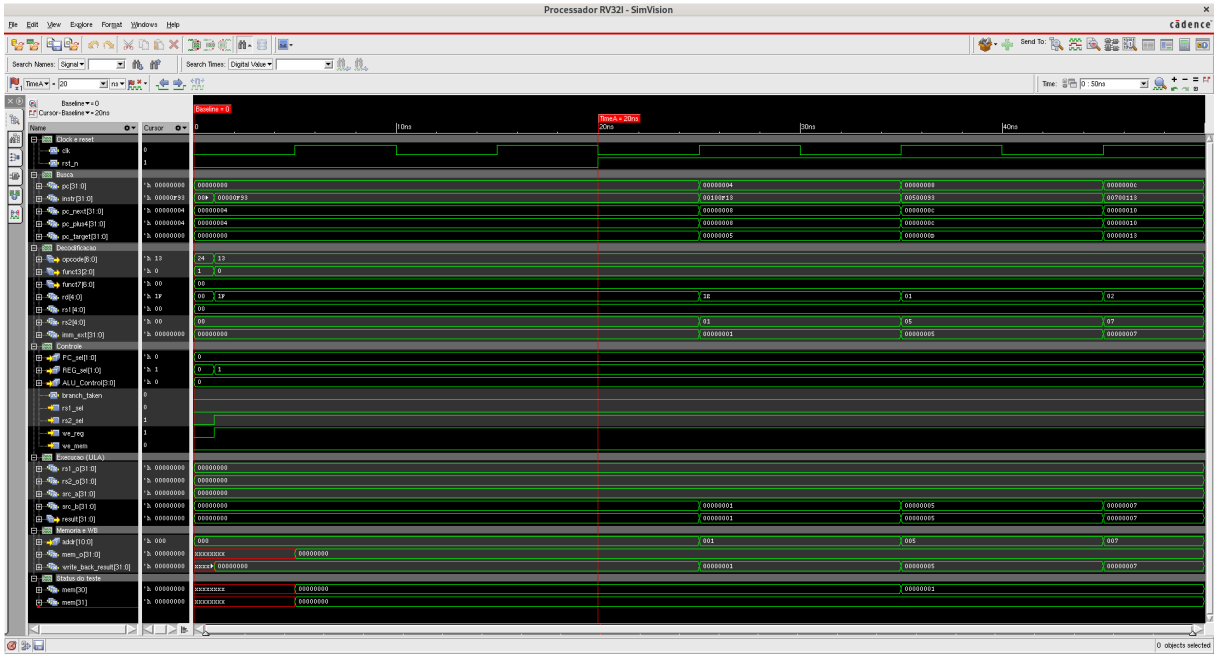


Figura 7: Fim do reset (programa 01). Em 20 ns `rst_n` sobe com o PC em `0x00000000`. A primeira instrução, `0x00000f93` (`addi x31, x0, 0`), já está na saída da memória de instruções, e o PC passa a avançar de 4 em 4.

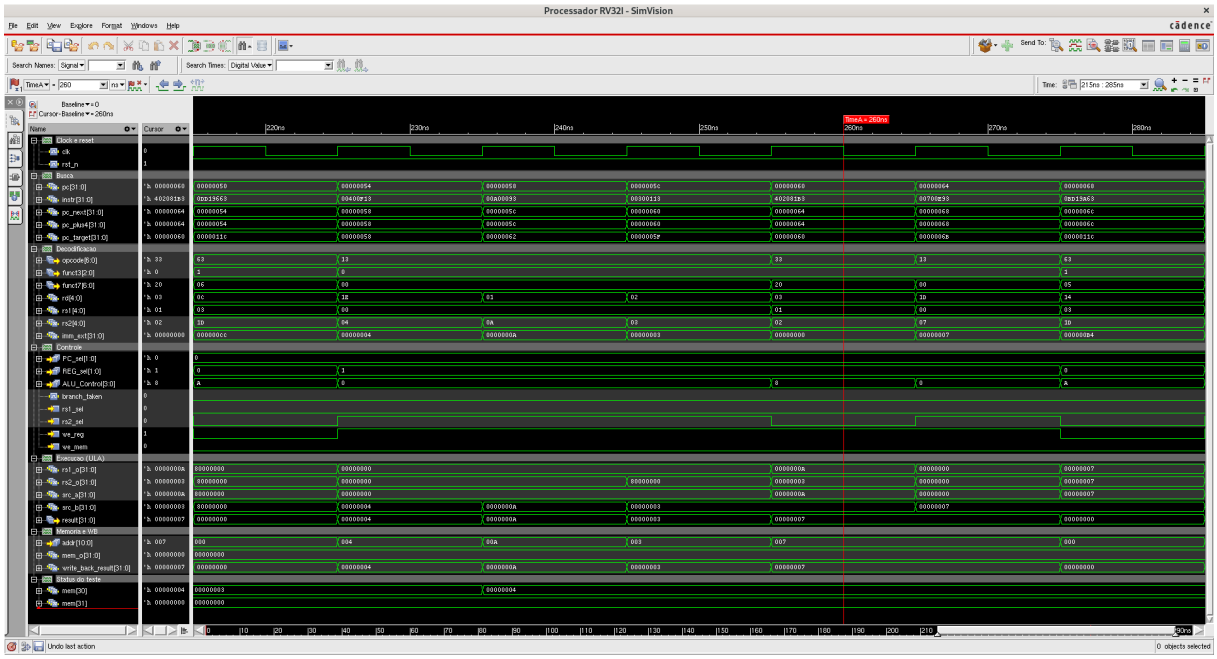


Figura 8: SUB (programa 01, 260 ns). A instrução `0x402081b3` é `sub x3, x1, x2`, no PC `0x60`, com `x1 = 0xA` e `x2 = 3`. Como `funct7 = 0x20`, o `alu_controller` gera o código de SUB (8) e a ULA devolve 7. `REG_sel = 01` manda o resultado para `x3`.

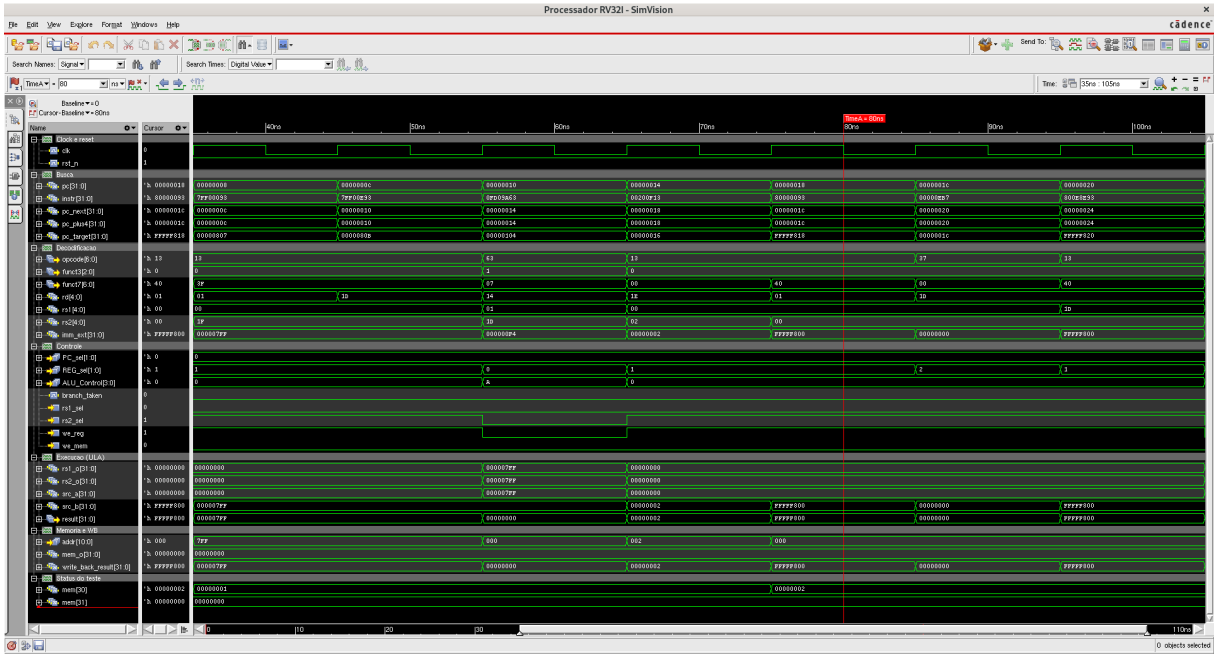


Figura 9: ADDI com imediato negativo (programa 02, 80 ns). `0x80000093` é `addi x1, x0, -2048`. O `imm_gen` estende o sinal e `imm_ext` vale `0xfffff800`. O campo `funct7` aparece como `0x40`, mas no tipo I esses bits fazem parte do imediato: `ALU_Control` fica em 0 (ADD) e o resultado é `0xfffff800`.

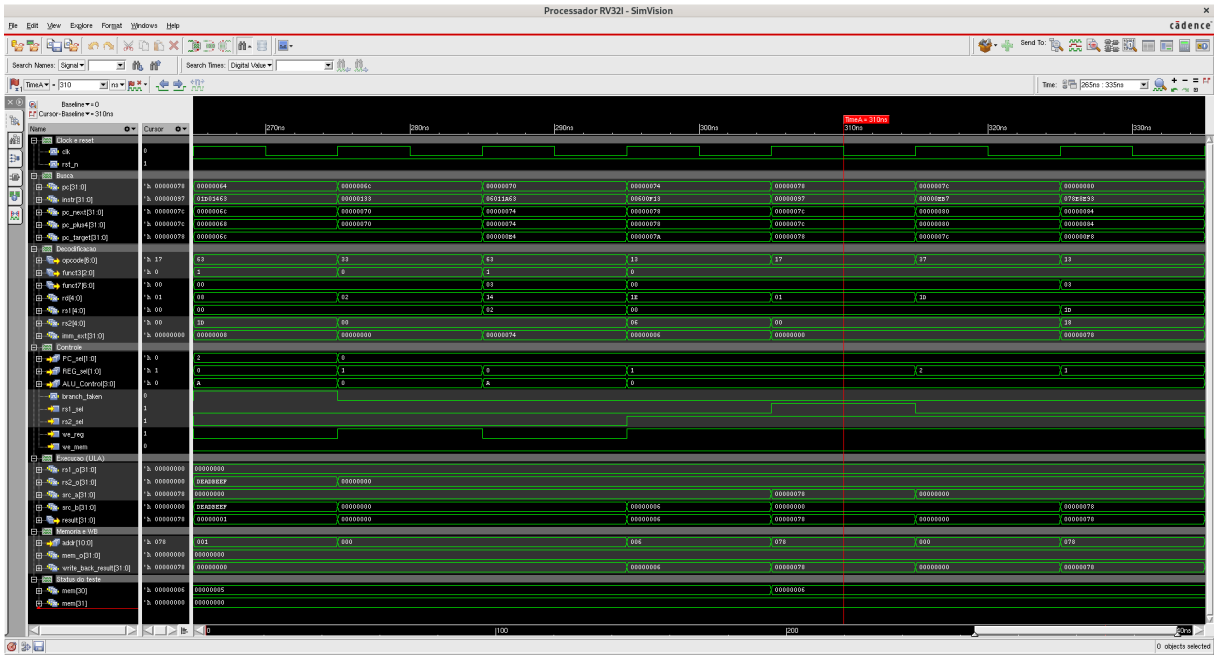


Figura 10: AUIPC (programa 05, 310 ns). `0x00000097` é `auipc x1, 0`, no PC `0x78`. Com `rs1_sel = 1`, a entrada `src_a` da ULA recebe o PC e o resultado é `0x78`. Era exatamente aqui que aparecia o erro do `PC + 2·imm`.

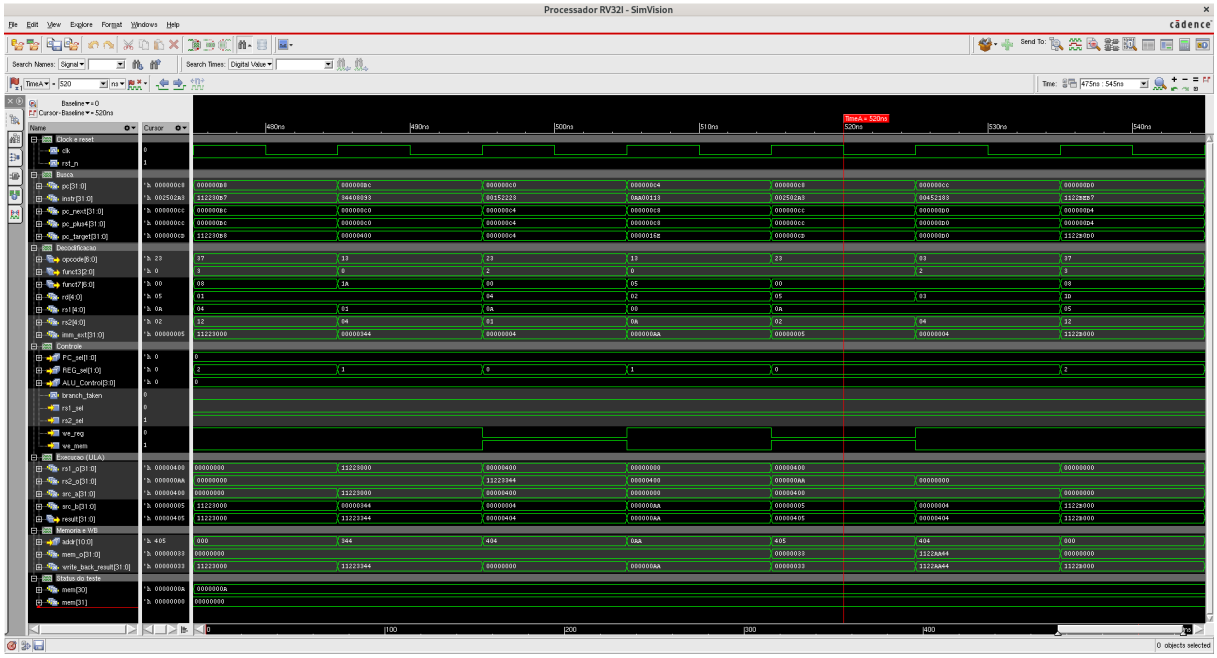


Figura 11: SB (programa 07, 520 ns). `0x002502a3` é `sb x2, 5(x10)`, com `x10 = 0x400` e `x2 = 0xAA`. A ULA calcula o endereço `0x405`, `we_mem` vai a 1 e, na borda de subida, só o byte desse endereço é escrito.

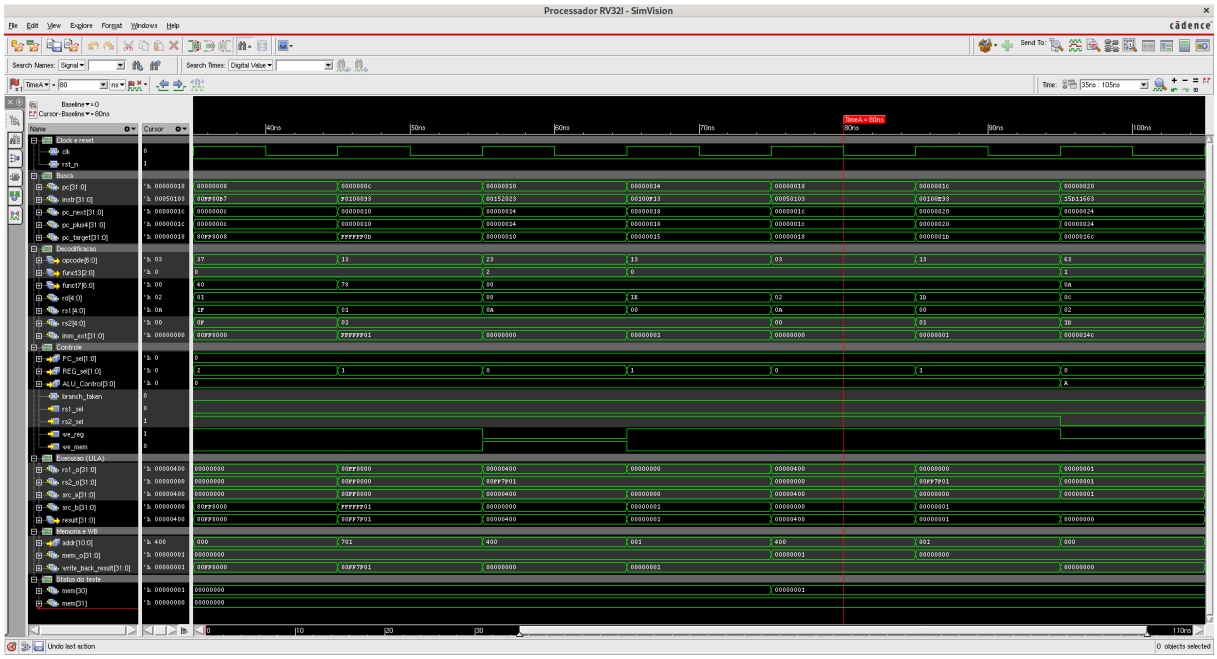


Figura 12: LB (programa 07, 80 ns). `0x00050103` é `lb x2, 0(x10)`, com endereço `0x400`. O byte guardado ali é `0x01`, então `mem_o` vale `0x00000001`, e `REG_sel = 00` grava esse valor em `x2`.

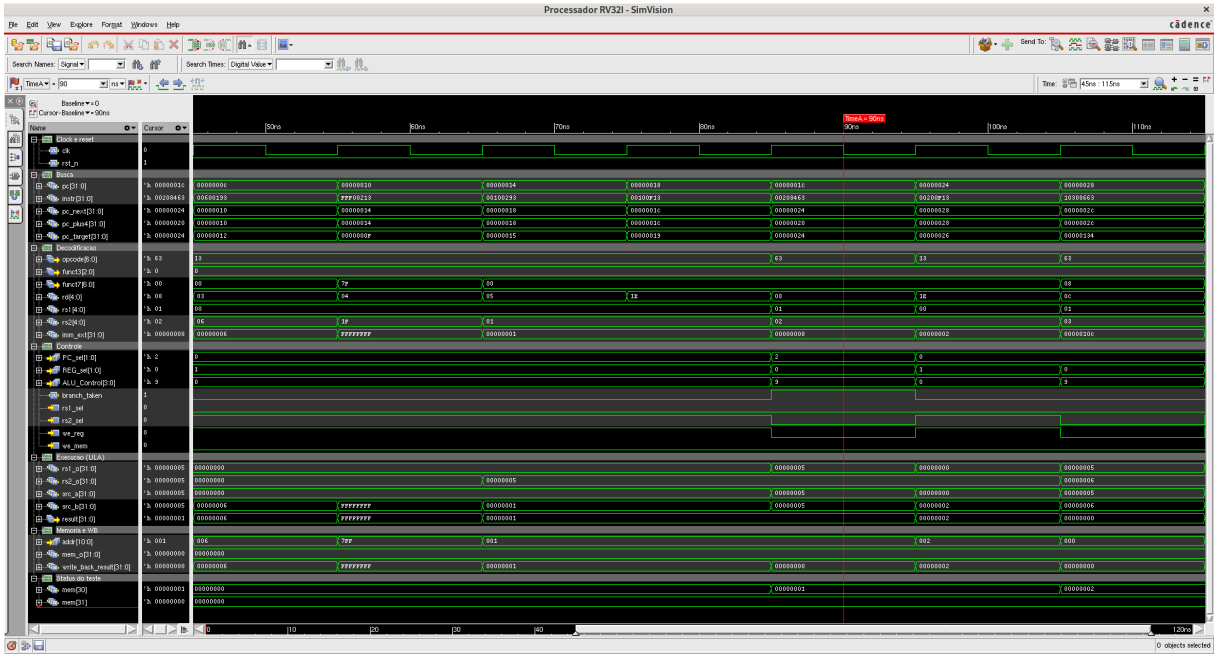


Figura 13: Desvio tomado (programa 08, 90 ns). `0x00208463` é `beq x1, x2, 8`, no PC `0x1c`. Os dois registradores valem 5, a ULA (código `9`, BEQ) põe `branch_taken = 1` e o `controller` coloca `PC_sel = 10`: o próximo PC é `0x24`.

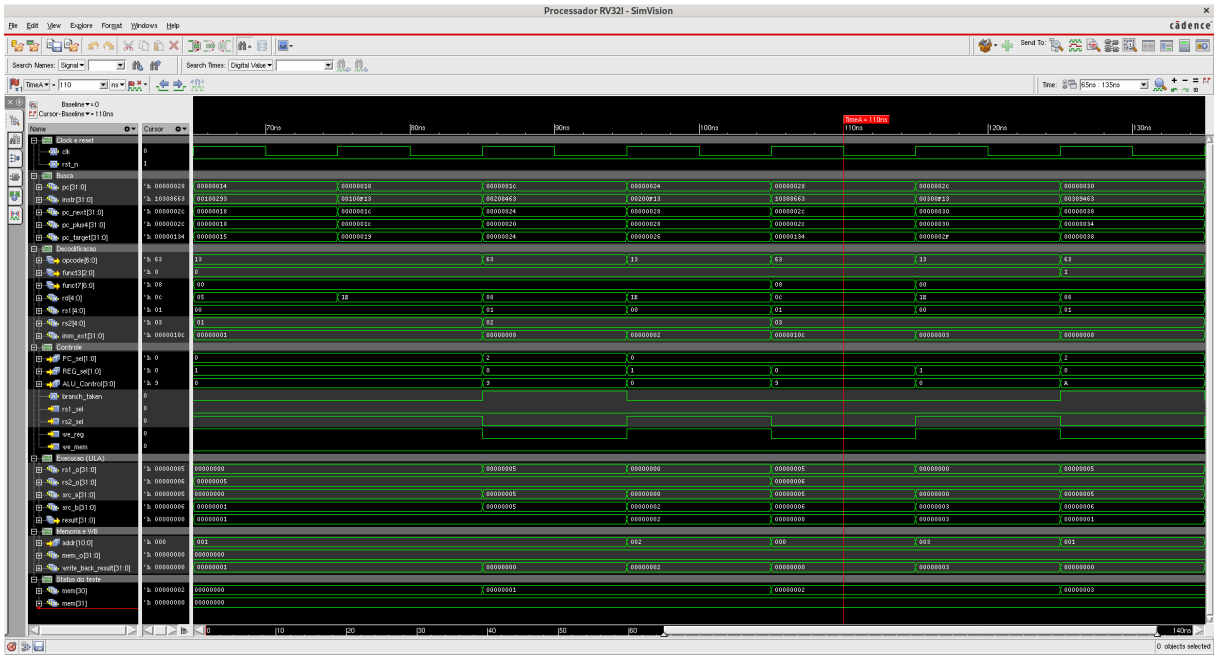


Figura 14: Desvio não tomado (programa 08, 110 ns). `0x10308663` é um `beq x1, x3` no PC `0x28`, com `x1 = 5` e `x3 = 6`. Como os valores são diferentes, `branch_taken` fica em 0, `PC_sel = 00` e a execução segue em `0x2c`.

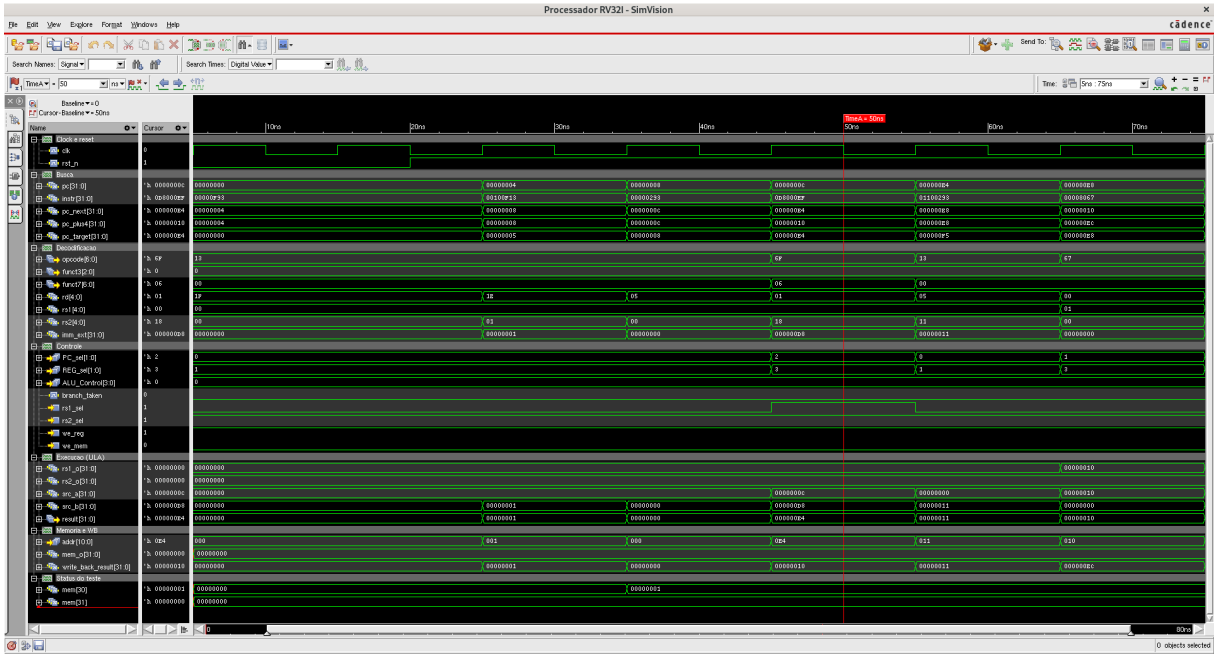


Figura 15: JAL (programa 09, 50 ns). `0x0d8000ef` é `jal x1, 0xd8`, no PC `0x0c`. O próximo PC é `0xe4` (`PC + imm`) e `REG_sel = 11` grava em `x1` o endereço de retorno, `0x10`.

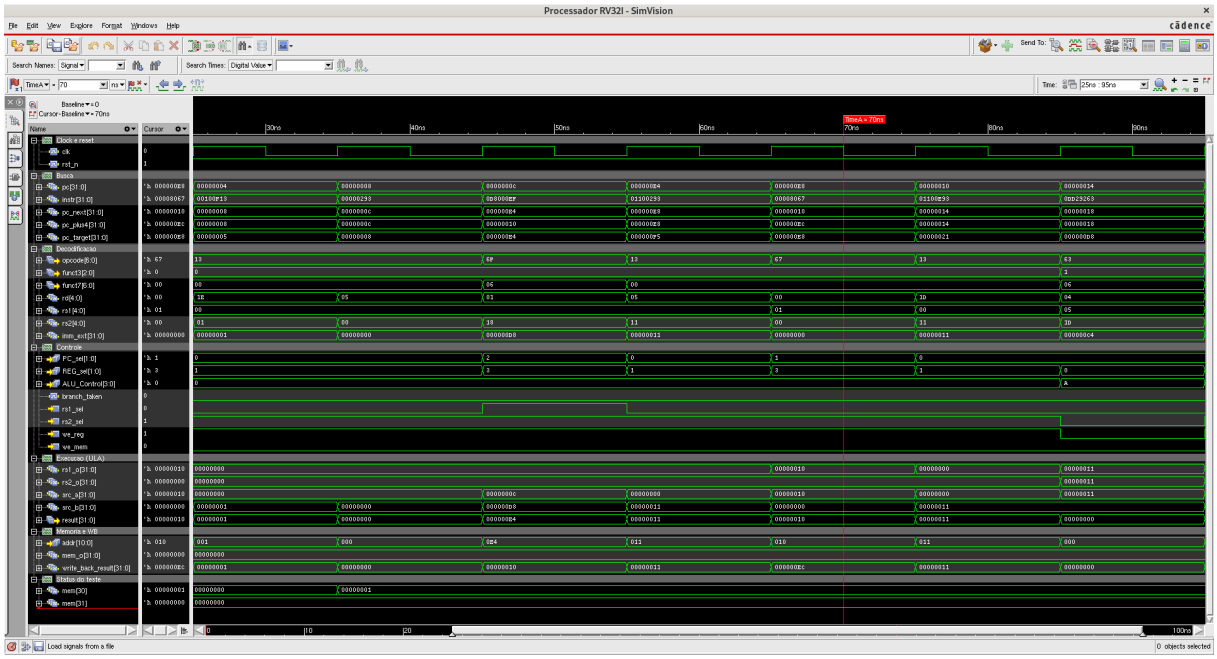


Figura 16: JALR (programa 09, 70 ns). `0x00008067` é `jalr x0, 0(x1)`, o retorno da sub-rotina, no PC `0xe8`. A ULA soma `x1 + 0`, o bit 0 é zerado e a execução volta para `0x10` (`PC_sel = 01`), logo depois do JAL da Figura 15.

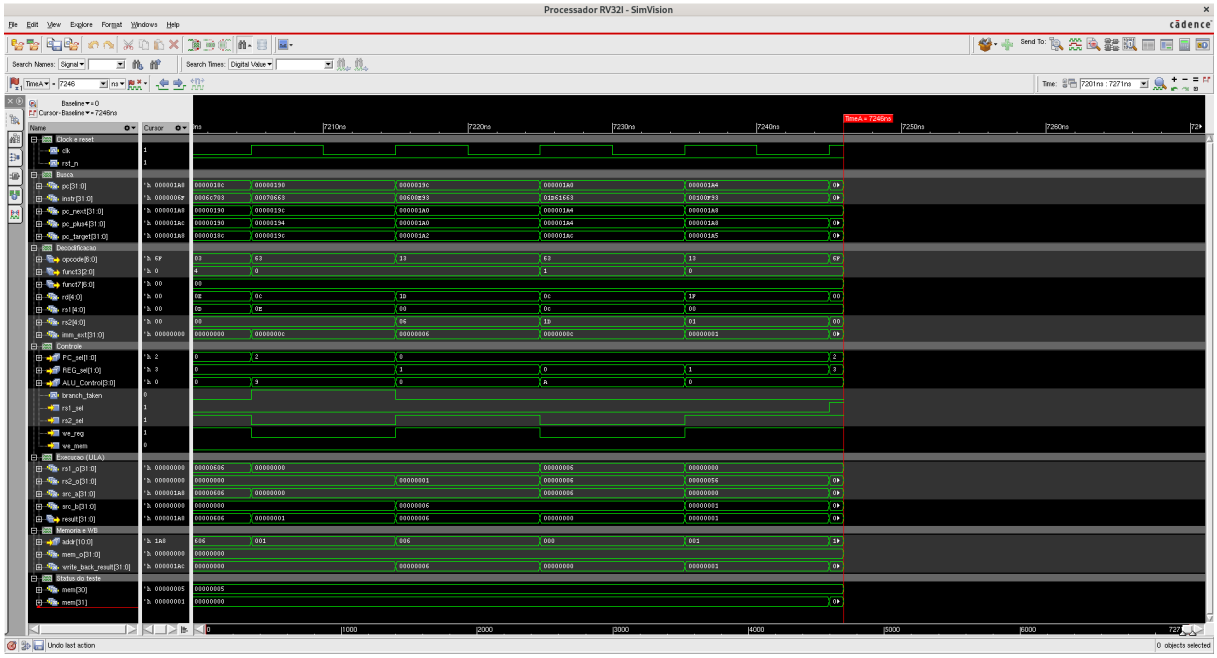


Figura 17: Fim do programa 10 (7246 ns). O PC chega ao laço final em `0x1a8` (`0x0000006f`, `j fim`). `x30 = 5` mostra que os cinco subtestes rodaram e `x31 = 1` indica que todos passaram.

6 Conclusão

O processador RV32I de ciclo único está completo e executa todas as instruções da Seção 2. Os 14 testbenches de bloco e os 10 programas passam no Xcelium, e em todos os programas o estado final dos registradores e o número de ciclos batem com o simulador de referência.

Os testes com programas completos foram os que mais ajudaram. Eles acharam três erros que os testes de bloco não pegavam: no AUIPC, nos acessos de byte e meia palavra e no tamanho da memória de dados. Cada bloco, sozinho, funcionava; os erros só apareceram com tudo ligado.

Tudo o que usamos está no repositório. O RTL e os testbenches ficam em `rtl/` e `tb/`, e os programas em `instructions/programs/`, com os registradores esperados em `.regs`. O código assembly e as ferramentas que geraram os testes estão em `instructions/arquivos_assembly/`, e os scripts de simulação e de formas de onda, em `sim/`.